formula in the advanced sections towards the end of the chapter.

4.5 Under the Hood - How Do Bloom Filters Work

Let's now delve into the details of Bloom filters implementation. A Bloom filter is made of two elements:

- An array of `m` elements.
- A set of `k` hash functions.

The array is (conceptually) an array of bits, each of which is initially set to `0`; each hash function outputs an index between `0` and `m-1`.

It is crucial to clarify as soon as possible that there is not a 1-to-1 correspondence between the elements of the array and the keys we add to the Bloom filter: rather, we will use `k` bits (and so `k` array's elements) to store each entry for the Bloom filter; `k` here is typically much smaller than `m`.

Note that `k` is a constant that we pick when we create the data structure, so each and every entry we add is stored using the same amount of memory, exactly `k` bits. With string values, this is pretty amazing: we can add arbitrarily long strings to our filter using a constant amount of memory, just `k` bits.

When we insert a new element key into the filter, we compute `k` indices for the array, given by the values $h_0(\texttt{key})$